Supplementary Notes: Normalizing Flows

Hong Qin, Old Dominion University

Fall 2026

About these supplementary notes

Normalizing flows were covered as a full lecture in the 16-week version of this course. In the 8-week format they are not part of the core Week 5 material and are provided here as optional reading. They are most useful as a companion to the flows-versus-diffusion comparison in the Week 5 lecture notes: flows are the deterministic, exact-likelihood cousins of diffusion models, and the probability flow ODE makes diffusion a special case of the continuous flows described at the end of this document.

Abstract

Normalizing flows are likelihood-based generative models built from invertible, differentiable transformations that map a simple base distribution to a complex data distribution. This note presents the core equations, layer constructions, and training procedures with detailed, step-by-step explanations and a boxed summary of continuous normalizing flows (CNFs).

Contents

1 Problem Setup and Goal **1**

2 Composing Simple Bijections **2**

3 Affine Coupling Layers (RealNVP/Glow-style) **2**

4 Autoregressive Flows (MAF vs. IAF) **3**

5 Invertible 1×1 Convolutions and ActNorm (Glow) **3**

6 Training: Exact Likelihood and Bits-per-Dimension **3**

7 Discrete Data and Dequantization **4**

CNF Box: Continuous Normalizing Flows **4**

8 Conditioning and Conditional Likelihoods **4**

9 Practical Stability Tips **4**

10 Minimal Pseudocode (Training and Sampling) **5**

11 Worked 2D Example (Two Coupling Layers) **5**

1 Problem Setup and Goal

We observe samples $x \in \mathbb{R}^D$ from an unknown data distribution $p_{\text{data}}(x)$. We choose a simple, tractable *base* density $p_Z(z)$ (e.g., standard Gaussian) and a bijective, differentiable map

$$f_\theta : \mathbb{R}^D \to \mathbb{R}^D, \qquad x = f_\theta(z), \quad z = f_\theta^{-1}(x). \tag{1}$$

Our aim is to model $p_X(x)$ exactly via change of variables:

$$p_X(x) = p_Z\Big(f_\theta^{-1}(x)\Big)\,\Big|\det J_{f_\theta^{-1}}(x)\Big| = p_Z(z)\,\left|\det J_{f_\theta}(z)\right|^{-1}, \tag{2}$$

where J_{f_θ} is the Jacobian of f_θ.

Remark (Two tractability requirements). To evaluate $\log p_X(x)$ in (2), we need: (i) f_θ invertible with a cheap inverse, and (ii) a cheap, exact computation of $\log|\det J_{f_\theta}(\cdot)|$.

Training by Maximum Likelihood. Given data $\{x^{(i)}\}_{i=1}^N$, we maximize

$$\mathcal{L}(\theta) \;=\; \frac{1}{N}\sum_{i=1}^{N} \log p_X\Big(x^{(i)}\Big) = \frac{1}{N}\sum_{i=1}^{N}\Big[\log p_Z\Big(z^{(i)}\Big) - \log\Big|\det J_{f_\theta}\Big(z^{(i)}\Big)\Big|\Big], \quad z^{(i)} = f_\theta^{-1}\Big(x^{(i)}\Big). \tag{3}$$

2 Composing Simple Bijections

We construct f_θ as a composition of K simple, invertible layers:

$$z_0 \equiv z \xrightarrow{f_1} z_1 \xrightarrow{f_2} \cdots \xrightarrow{f_K} z_K \equiv x, \tag{4}$$

so $f_\theta = f_K \circ \cdots \circ f_1$. The log-density factorizes as

$$\log p_X(x) = \log p_Z(z_0) \;-\; \sum_{k=1}^{K} \log\left|\det J_{f_k}(z_{k-1})\right|. \tag{5}$$

Remark. The sum of log-determinants in (5) is the core engineering focus: each f_k must provide a cheap inverse and a cheap $\log|\det J_{f_k}|$.

3 Affine Coupling Layers (RealNVP/Glow-style)

Split variables via a mask into two parts $x = (x_a, x_b)$. Define

Definition (Forward transform).

$$y_a = x_a, \tag{6}$$
$$y_b = x_b \odot \exp(s_\theta(x_a)) + t_\theta(x_a), \tag{7}$$

where s_θ and t_θ are small neural networks. Here, x_a acts as a *reference*: it is copied directly to y_a, and it tells the networks how much to *stretch* (scale via $\exp(s_\theta)$) and *shift* (via t_θ) the other half x_b.

Remark (Intuition about splitting). Why do we split? If we tried to transform all components of x at once, the Jacobian would be a full $D \times D$ matrix and computing its determinant would cost $O(D^3)$. By splitting, we only transform one half based on the other, so the Jacobian becomes triangular. The determinant of a triangular matrix is just the product of its diagonal entries, making it very cheap to compute.

Remark (Plain English). Think of it like this: half of the variables "stay put" (x_a), and that frozen half whispers instructions (how much to stretch/shift) to the other half. This allows flexible nonlinear transformations while still being invertible and mathematically simple.

Inverse (closed form). Because the forward mapping is affine in x_b, inversion is trivial:

$$x_a = y_a, \qquad x_b = (y_b - t_\theta(y_a)) \odot \exp(-s_\theta(y_a)). \tag{8}$$

Log-determinant (triangular Jacobian). The Jacobian matrix is triangular: the top-left block is identity (since $y_a = x_a$), and the bottom-right block is diagonal (due to elementwise scaling of x_b). Hence,

$$\log \left| \det \frac{\partial y}{\partial x} \right| = \sum_j s_\theta(x_a)_j. \tag{9}$$

Remark (Key advantage). Because the log-determinant is just the sum of the scale outputs s_θ, it is extremely cheap to compute (linear time in the number of dimensions). This is why coupling layers are the workhorse of normalizing flows.

Remark (Design pattern). Alternate masks (or use permutations/invertible 1×1 convolutions) across layers so both halves influence each other after several blocks. Clamp s_θ (e.g., with tanh) to improve numerical stability.

4 Autoregressive Flows (MAF vs. IAF)

Choose an ordering of dimensions and define an affine autoregressive transform:

$$y_i = \mu_i(x_{<i}) + \sigma_i(x_{<i})\, x_i, \qquad i = 1, \ldots, D; \quad \sigma_i > 0. \tag{10}$$

The Jacobian is lower triangular, so

$$\log \left| \det \frac{\partial y}{\partial x} \right| = \sum_{i=1}^{D} \log \sigma_i(x_{<i}). \tag{11}$$

- **MAF** (Masked Autoregressive Flow): fast *density evaluation* (inverse is parallelizable), slower sampling.
- **IAF** (Inverse Autoregressive Flow): fast *sampling* (forward is parallelizable), slower density evaluation.

5 Invertible 1×1 Convolutions and ActNorm (Glow)

For image tensors ($H \times W \times C$), apply a per-location invertible linear map across channels:

$$y(h, w, :) = W\, x(h, w, :), \qquad W \in \mathbb{R}^{C\times C},\ \det W \neq 0. \tag{12}$$

Then

$$\log \left| \det \frac{\partial y}{\partial x} \right| = (H \cdot W) \, \log |\det W|. \tag{13}$$

Stability tip: Parametrize $W = PLU$ (fixed permutation P, learnable L, U with positive diagonals) so $\log |\det W| = \sum_i \log U_{ii}$.

ActNorm (per-channel affine). $y = s \odot x + b \;\Rightarrow\; \log |\det| = (H \cdot W) \sum_c \log |s_c|$.

6 Training: Exact Likelihood and Bits-per-Dimension

For a datapoint x:

Step 1. Invert: compute $z = f_\theta^{-1}(x)$ by applying inverses of layers $f_K^{-1}, \ldots, f_1^{-1}$.

Step 2. Accumulate log-det: sum $\sum_k \log |\det J_{f_k}(z_{k-1})|$ during inversion.

Step 3. Base logprob: for standard Gaussian,

$$\log p_Z(z) = -\frac{1}{2} \left(\|z\|_2^2 + D \log(2\pi) \right). \tag{14}$$

Step 4. Likelihood: $\log p_X(x) = \log p_Z(z) - \sum_k \log |\det J_{f_k}|$.

Step 5. Loss: minimize $-\log p_X(x)$ over the batch with Adam.

Bits-per-Dimension (images).

$$\text{bpd} = -\frac{1}{D \log 2} \, \mathbb{E}_x \left[\log p_X(x) \right], \qquad D = H \cdot W \cdot C. \tag{15}$$

7 Discrete Data and Dequantization

Images are discrete (e.g., 8-bit). Flows require continuous inputs. *Uniform dequantization*:

$$u \sim \text{Unif}([0,1)^D), \qquad x' = \frac{x + u}{256}, \tag{16}$$

and we train on x'. *Variational dequantization* learns a conditional $q_\phi(u \mid x)$ (often another small flow) for a tighter likelihood bound.

Remark. Dequantization prevents assigning probability mass to measure-zero grid points and yields a valid continuous objective that corresponds to a bound on the discrete likelihood.

CNF Box: Continuous Normalizing Flows

Continuous Normalizing Flows (CNF)

Define a time-indexed state $z(t)$ via the ODE

$$\frac{dz(t)}{dt} = g_\theta(z(t), t), \qquad z(0) \sim p_Z, \quad x = z(T). \tag{17}$$

The *instantaneous change of variables* gives:

$$\frac{d}{dt} \log p(z(t)) = -\mathrm{tr}\left(\frac{\partial g_\theta}{\partial z}\right). \tag{18}$$

Integrating,

$$\log p_X(x) = \log p_Z(z(0)) - \int_0^T \mathrm{tr}\left(\frac{\partial g_\theta}{\partial z}\right) dt. \tag{19}$$

Practical estimators. Use Hutchinson's trick for traces: $\mathrm{tr}(A) = \mathbb{E}_v\left[v^\top A v\right]$ with v Rademacher/Gaussian, computable via Jacobian-vector products. For gradients through the solver, use the adjoint method.

Trade-off. CNFs relax triangular/Jacobian structure constraints but add ODE solve cost; accuracy *vs* speed tunes via solver tolerances/steps.

8 Conditioning and Conditional Likelihoods

To model $p(x \mid c)$, parameterize flows with context c:

$$y_b = x_b \odot \exp\big(s_\theta(x_a, c)\big) + t_\theta(x_a, c), \tag{20}$$

or use FiLM/concatenation/cross-attention. Train by maximizing $\sum_i \log p_\theta(x^{(i)} \mid c^{(i)})$; sample via $z \sim p_Z$, then $x = f_\theta(z; c)$.

9 Practical Stability Tips

- Clamp scaling outputs s_θ (e.g., tanh) to avoid extreme exponentials.
- Use PLU factorization for invertible 1×1 convolutions.
- Track log-dets in *float64* if needed; use log-sum-exp tricks.
- Regularize with weight norm/spectral norm on s_θ, t_θ networks when helpful.

10 Minimal Pseudocode (Training and Sampling)

Training (one minibatch)

Listing 1: Flow training loop (schematic).

```
loss = 0.0
```

```
for x in batch: # x: data sample
    z = x
    logdet = 0.0
    # Invert through layers: x -> z
    for k in reversed(range(K)): # f_K^{-1}, ..., f_1^{-1}
        z, ld = f_k.inverse_and_logdet(z) # returns z_{k-1}, log|det J_{f_k}|
        logdet += ld
    base_lp = base_logprob(z) # e.g., standard Gaussian
    loss += -(base_lp - logdet) # NLL = -log p_X(x)
loss /= len(batch)
loss.backward()
optimizer.step()
```

Sampling

Listing 2: Flow sampling (schematic).

```
z = sample_base(m) # m samples from p_Z
x = z
for k in range(K): # forward: z -> x
    x = f_k.forward(x)
return x
```

11 Worked 2D Example (Two Coupling Layers)

Let $x = (x_1, x_2)$. **Layer 1** (mask a=$\{1\}$):

$$y_1 = x_1, \tag{21}$$

$$y_2 = x_2 \exp\big(s_1(x_1)\big) + t_1(x_1), \tag{22}$$

so $\log|\det J_1| = s_1(x_1)$.

(23)

Layer 2 (swap roles; mask a=$\{2\}$):

$$z_2 = y_2, \tag{24}$$

$$z_1 = y_1 \exp\big(s_2(y_2)\big) + t_2(y_2), \tag{25}$$

so $\log|\det J_2| = s_2(y_2)$.

(26)

Total log-det:

$$\log|\det J| = s_1(x_1) + s_2\big(y_2(x)\big). \tag{27}$$

Finally, score $z = (z_1, z_2)$ via (14) and assemble $\log p_X(x)$ using (5).

References (classic exemplars)

- L. Dinh, J. Sohl-Dickstein, S. Bengio (2017). *Density Estimation using Real NVP*. ICLR 2017, arXiv:1605.08803.
- D. P. Kingma et al. (2016). *Improving Variational Inference with Inverse Autoregressive Flow (IAF)*. NeurIPS 2016, arXiv:1606.04934; G. Papamakarios, T. Pavlakou, I. Murray (2017). *Masked Autoregressive Flow for Density Estimation (MAF)*. NeurIPS 2017, arXiv:1705.07057.
- D. P. Kingma, P. Dhariwal (2018). *Glow: Generative Flow with Invertible* 1×1 *Convolutions*. NeurIPS 2018, arXiv:1807.03039.
- R. T. Q. Chen, Y. Rubanova, J. Bettencourt, D. Duvenaud (2018). *Neural Ordinary Differential Equations*. NeurIPS 2018, arXiv:1806.07366.
- W. Grathwohl, R. T. Q. Chen, J. Bettencourt, I. Sutskever, D. Duvenaud (2019). *FFJORD: Free-form Continuous Dynamics for Scalable Reversible Generative Models*. ICLR 2019, arXiv:1810.01367.